Linear Models for News Article Classification

Matteo Sgobba
Politecnico di Torino
Student id: s361177
matteo.sgobba@studenti.polito.it

***Abstract*—This report investigates a news classification task using TF-IDF representations and linear classifiers. Models are tuned through a progressive hyperparameter search and evaluated using macro-averaged f_1 score. The results demonstrate that simple linear approaches achieve competitive performance on unseen data despite class imbalance and high dimensionality.**

I. Problem overview

The task addressed in this project consists in the multi-class classification of online news articles into a predefined set of semantic categories, based on their textual content and associated metadata.

The dataset provided for the project contains approximately 100,000 news articles collected from multiple international news outlets and it is partioned in two parts:

- a *development* set, containing 79,997 records and characterised by the identifier (*id*), *source*, title of the article (*title*), text of the article (*article)*, page rank related to the source (*page_rank*), publication date and time (*timestamp*) and the associated label (*label*).
- an *evaluation* set, containing 20,000 records structured in the same way as the development set, but without labels.

The target variable label encodes seven news categories as follows: International News (0), Business (1), Technology (2), Entertainment (3), Sports (4), General News (5), and Health (6). The distribution of the target labels is shown in Figure 1.

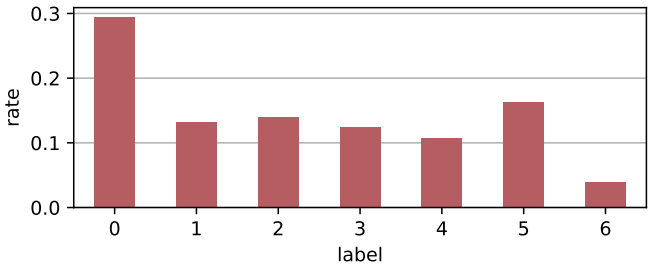

Fig. 1. Distribution of the labels among the articles

The dataset exhibits a moderate class imbalance, with International News (label 0) being the most frequent category, while Health (label 6) is substantially underrepresented. This imbalance is particularly relevant given that macro-averaged f_1 score is used as the evaluation metric: rare classes contribute equally to the final score, making their correct classification more challenging and increasing performance variance. This aspect is expected to negatively affect the overall macro-f_1, especially for label 6, and must be taken into account when interpreting results.

An inspection of missing values shows that the dataset is largely complete. Only a negligible fraction of records exhibits missing content in the title or article fields (approximately 0.125% each), which does not materially affect downstream processing. A more relevant issue concerns the timestamp attribute: while present for most articles, a non-negligible subset contains placeholder values (i.e., `"0000-00-00 00:00:00"`). The proportion of such placeholders varies substantially across categories, ranging from about 22% for label 2 to roughly 40% for label 6, suggesting heterogeneity in data collection or publication practices across domains.

The distribution of article lengths, measured in word count, was analyzed across labels (Figure 2).

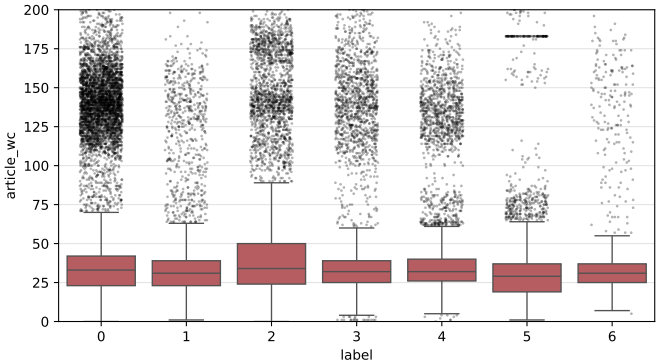

Fig. 2. Article word count distribution by label

Article lengths are reasonably consistent across categories, with comparable medians and interquartile ranges. The overall median for both title and article lenghts is 7 and 32, respectively. However, the distributions are characterized by a large number of outliers, corresponding to unusually short or long articles.

In particular, a substantial number of extreme low-length outliers (word count equal to 1) were observed for Technology (label 2) and General News (label 5). Manual inspection revealed that these cases correspond to placeholder tokens (e.g., "\N") rather than meaningful textual content. This observation motivated specific preprocessing decisions aimed at filtering such artifacts before model training.

With respect to news sources, the dataset includes articles originating from a wide and heterogeneous set of publishers;

Table I reports the top 10 most frequent sources, ranked by the number of associated articles. Notably, the evaluation set includes 116 sources that do not appear in the development set, yet accounting for only $\approx 0.63\%$ of evaluation samples.

TABLE I
TOP 10 SOURCES BY ARTICLE COUNT

Soruce	N. articles
Yahoo	15850
Reuters	11915
BBC	7971
New	6430
Washington	3383
RedNova	2701
Boston	1807
CNN	1802
CNET	1369
Topix.Net	1331

Finally, analysis of the *page_rank* attribute reveals a strong dominance of a single value (rank = 5) across nearly all categories. This lack of variability indicates limited discriminative power for this feature, which is therefore expected to contribute marginally to classification performance and is treated accordingly in the modeling phase.

II. PROPOSED APPROACH

A. Preprocessing

The preprocessing pipeline is designed to produce a clean and consistent representation of each article while accounting for the different representational requirements of the linear classifiers described in the next section. The overall goal is to preserve informative textual patterns while ensuring compatibility with high-dimensional sparse learning.

Following exploratory analysis, articles containing only placeholder tokens (the string "\N") were filtered out, as they carry no semantic information and would introduce noise. Two auxiliary attributes, *title_wc* and *article_wc*, were also computed to capture basic length-related properties of titles and article bodies. All textual fields were further cleaned by applying HTML unescaping, in order to remove encoded entities.

A unified textual field, denoted as *text*, was constructed by concatenating source, title, and article content. Including the source directly within the text allows the model to implicitly capture publisher-specific lexical and stylistic patterns without introducing an explicit categorical variable. After constructing this field, duplicate articles sharing the same *text* field and label were removed, resulting in the elimination of 1,368 duplicated instances.

The preprocessing pipeline handles temporal information by introducing a binary indicator (*ts_missing*) to encode placeholder timestamps. When available, timestamps were decomposed into *year*, *month*, *dayofweek*, and *hour*, while the raw timestamp itself was excluded from the model input, as it is not suitable for linear learning.

The final document representation is based on Term Frequency–Inverse Document Frequency (TF-IDF) features. Given a term *t* and a document *d*, the TF-IDF is defined as:

$$\mathrm{TF-IDF}(t,d) = \mathrm{tf}(t,d) \cdot \log\left(\frac{N}{\mathrm{df}(t)}\right)$$

where *tf(t, d)* is the frequency of term *t* in document *d*, *df(t)* denotes the document frequency of *t*, and *N* is the total number of documents. This representation emphasizes discriminative terms while down-weighting frequent ones, a property that is particularly effective for topic-based text classification.

Two TF-IDF configurations were employed. Word-level TF-IDF features with an n-gram range of (1, 2), capturing both unigrams and bigrams, were used for all classifiers, and constituted the sole representation adopted for Logistic Regression. In addition, character-level TF-IDF features based on n-grams of length (3,5) were combined with LinearSVC and SGD-Classifier. This choice is motivated by prior evidence showing that margin-based linear classifiers are particularly well suited to high-dimensional and sparse text representations, where many weakly informative features can be effectively combined to improve class separability [1]. In contrast, probabilistic linear models such as Logistic Regression tend to benefit less from character-level representations, as the induced feature redundancy and correlation do not necessarily translate into improved likelihood estimation [2]. For word-level TF-IDF, the document frequency thresholds were set to *min_df* = 2 and *max_df* = 0.90, while for character-level TF-IDF the same minimum frequency was used with a slightly higher maximum threshold *max_df* = 0.95, to account for the more granular nature of character n-grams.

In addition, english stop words were removed and sublinear term frequency scaling was applied during vectorization, replacing raw term frequencies with $1 + \log(\mathrm{tf})$. This transformation reduces the influence of very frequent terms within a document, mitigating dominance effects and improving robustness.

All TF-IDF vectors were L2-normalized, which stabilizes optimization in high-dimensional sparse spaces while retaining potentially informative features; L1 normalization was therefore avoided, as it would enforce sparsity by zeroing out potentially informative features. The few numerical features were scaled using standard scaling without mean centering, which is recommended for sparse representations [3]. All preprocessing steps were fitted exclusively on the development set and applied unchanged to the evaluation set to prevent information leakage.

B. Model selection

As already anticipated, given both the scale of the dataset and the intrinsic sparsity of bag-of-words representations, the modeling strategy focused on linear classifiers operating on TF-IDF features. This type of models is well known to provide a favorable trade-off between expressive power, robustness, and computational efficiency in text classification settings [4].

Linear decision boundaries are particularly well suited to vector space representations of documents, where relevant information is distributed across a large number of sparse features. In contrast, non-linear models such as decision trees or instance-based methods (e.g., k-nearest neighbors) tend to suffer from the curse of dimensionality, are more sensitive to noise, and scale poorly when the number of features largely exceeds the number of samples [1], [5].

The following classifiers were selected:

- **Logistic Regression:** Logistic Regression is a probabilistic linear classifier that models class posterior probabilities through a linear decision function followed by a softmax transformation. When combined with TF-IDF features, it provides a strong and well-calibrated model for multi-class text classification, offering both interpretability and stable optimization in high-dimensional sparse spaces [4], [6]. Its convex objective function ensures reliable convergence and makes it particularly suitable for medium-to-large textual datasets represented in vector space form [2].
- **Linear Support Vector Classification:** Linear Support Vector Machines learn a maximum-margin hyperplane by minimizing a hinge-loss objective, focusing on separating classes with the largest possible margin. This property makes LinearSVC particularly effective as well in high-dimensional settings where documents are represented by sparse TF-IDF vectors, as is common in text classification tasks with bag-of-words representations [1], [6].
- **Stochastic Gradient Descent Classifier:** The SGDClassifier was selected to serve as a scalable baseline for comparison. It simulates the behaviour of the two main models chosen for this task by optimizing linear classification objectives—such as logistic loss or hinge loss—using stochastic gradient descent. This allows a more efficient training on large datasets with high dimensionality. While not always achieving the same level of performance as more carefully optimized solvers, SGD-based classifiers provide a good approximation of linear models at a significantly lower computational cost [2], [7].

To address the class imbalance observed in the dataset—especially for the least frequent class (label 6)—all models were trained by setting the parameter *class_weight* to *balanced*, as also suggested by sklearn documentation [8]. This choice ensures that each class contributes proportionally to the optimization objective and prevents bias toward dominant categories. The resulting class weights computed on the development set are reported in Table II.

TABLE II
WEIGHTS ASSIGNED TO ALL THE LABELS

0	1	2	3	4	5	6
0.4842	1.0727	1.0226	1.1534	1.3203	0.8820	3.7530

As expected, rarer classes receive higher weights, while more frequent ones are down-weighted.

In practice, this weighting scheme is handled automatically by the learning algorithms with the balanced setting, hence no manual intervention is required during training.

This strategy allows all models to better account for under-represented classes while preserving a unified and comparable training setup across classifiers.

The optimal hyperparameter configurations for all models were identified through a multi-step Grid Search procedure on the development set, as detailed in the following section.

C. Hyperparameters tuning

Hyperparameters tuning aims to identify robust configurations for each classifier while keeping the computational cost manageable. To this end, all preprocessing-related parameters were fixed in advance, as discussed in the previous section, allowing the search to focus exclusively on the most influential model-specific hyperparameters.

The tuning process adopts a stepwise Grid Search strategy for the three models. Initial runs explored relatively broad ranges for the regularization parameter C, which were then progressively refined by narrowing the search space around the best-performing values identified in earlier steps. This progressive refinement enables a targeted exploration of the hyperparameter space while limiting computational overhead. This approach was particularly important for LinearSVC, whose training time proved to be substantially higher than that of the other models, on the order of hours.

A slightly different strategy applies to SGDClassifier, given its role as a scalable baseline. The learning rate α was explored over a fixed logarithmic grid with constant spacing between consecutive values. The tuning process proceeded by identifying the best-performing value and verifying that no neighboring values yielded superior performance, rather than progressively shrinking the search interval.

Model evaluation during hyperparameters tuning relies on 3-fold cross-validation. This choice allowed the entire development set to be used for both training and validation, avoiding an explicit train–validation split and yielding more reliable estimates of generalization performance. For each Grid Search run, the optimal configuration was selected based on the highest macro-averaged f_1 score across the folds.

The complete set of hyperparameters considered for each model is reported in Table III.

TABLE III
HYPERPARAMETERS CONSIDERED FOR MODEL SELECTION

Model	Parameter	Values
Logistic Regression	C	{1, 2, 3, 4, 5}
	solver	{lbfgs, saga}
LinearSVC	C	{0.01, 0.05, 0.1, 0.15, 0.2, 0.5, 1}
SGDClassifier	α	{1e-6, 5e-6, 1e-5, 5e-5, 1e-4, 5e-4, 1e-3, 5e-3}
	loss	{hinge, log_loss}

III. Results

To identify the best-performing configurations, a total of two tuning runs were conducted for each of the three models.

For Logistic Regression, the best configuration was obtained with {*model__C=2, model__solver='saga'*}, achieving a cross-validation macro-f_1 score of ≈ 0.7053. LinearSVC achieved its best performance with {*model__C = 0.1*}, yielding a higher cross-validation macro-f_1 score of ≈ 0.7109, corresponding to an improvement of approximately 0.0056 (+0.8%) over Logistic Regression. For SGDClassifier, the best configuration was {*model__alpha* = 1×10^{-5}, *model__loss = 'hinge'*}, with a cross-validation macro-f_1 score of ≈ 0.6888, showing a noticeable performance gap with respect to the other two approaches.

Although the difference between Logistic Regression and LinearSVC on the development set is relatively small, a clearer separation emerges when considering the public leaderboard evaluation. Logistic Regression achieved a macro-f_1 score of 0.728, while LinearSVC reached 0.735, resulting in a substantial improvement and securing a position within the top 25 of the leaderboard. These results suggest that the selected configurations generalize well to unseen data and that no evident overfitting occurred during model selection.

For completeness, Figure 3 illustrates the cross-validation macro-f_1 score as a function of the regularization parameter C for LinearSVC, selected as the best-performing model.

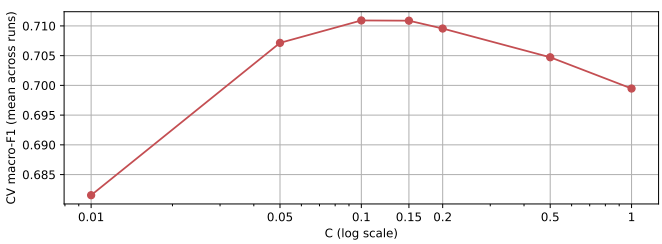

Fig. 3. LinearSVC: CV macro-averaged f_1 vs C

Similarly, the SGDClassifier configuration achieved a public leaderboard macro-f_1 score of ≈ 0.698, which is also higher than its corresponding cross-validation performance. Despite this improvement, however, the gap with respect to Logistic Regression and LinearSVC remains substantial, confirming the role of SGDClassifier as a baseline approach in this setting.

As an additional point of comparison, a naïve baseline based on random guessing was evaluated. This approach achieved a cross-validation macro-f_1 score of ≈ 0.1417 and a public leaderboard score of 0.150, confirming that all considered models substantially outperform trivial prediction strategies.

Finally, additional experimental runs were conducted without enforcing reproducibility constraints (i.e., without fixed random seeds). These runs yielded higher scores, such as 0.707 for SGDClassifier and 0.737 for LinearSVC, the latter corresponding to a potential top 10 placement. However, to ensure reproducibility and methodological consistency, these results were not considered for the final submission.

IV. Discussion

The results indicate that the proposed approach is effective and robust for the task. Both Logistic Regression and LinearSVC achieved strong and comparable performances on the public leaderboard, consistently outperforming both the naïve random-guessing baseline and the SGD-based baseline, even after tuning. Notably, the leaderboard scores of the models are slightly higher than their corresponding cross-validation scores on the development set, suggesting good generalization capabilities and no evident signs of overfitting.

A closer inspection of the public leaderboard further contextualizes these results. At the time of writing, the top scores range from 0.746 to 0.735 for the twenty-fifth position, corresponding to a narrow gap of about 0.011 (approximately 1.5%). Within this range, the proposed solution is well aligned with the leading approaches, indicating that the remaining margin for improvement is present but relatively limited.

Several factors may have constrained performance. As anticipated during preprocessing, one of the main challenges of the task is the presence of class imbalance. Despite the use of balanced class weights, this imbalance inevitably affects the macro-averaged f_1 score. At the same time, this evaluation metric is the most informative and realistic one in this setting, as it assigns equal importance to all classes and therefore provides a stricter and more reliable assessment of model performance under imbalance conditions [9].

Another intrinsic limitation arises from the extremely high dimensionality of the feature space induced by word- and character-level TF-IDF representations. While this choice is well suited for linear models, it results in substantial computational costs and long training times, particularly during hyperparameters tuning, and motivated the adoption of a conservative and highly targeted tuning strategy.

To conclude, potential directions for further improvement can be outlined:

- More extensive hyperparameter optimization: the tuning process focused on a limited set of parameters and relied on fixed preprocessing choices to control computational cost. A more exhaustive Grid Search, including additional parameters and a systematic tuning of TF-IDF preprocessing settings, could potentially lead to modest performance gains.
- Adoption of embedding-based models: modern neural architectures based on contextual embeddings, such as BERT, have become the standard for many natural language processing tasks. By leveraging deep bidirectional representations and contextualized token embeddings, such models are able to capture semantic and syntactic information beyond the capabilities of bag-of-words approaches, often resulting in significantly improved performance on text classification benchmarks [10]. However, these gains come at the cost of increased model complexity and computational requirements.

REFERENCES

[1] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in *Machine Learning: ECML-98* (C. Nédellec and C. Rouveirol, eds.), (Berlin, Heidelberg), pp. 137–142, Springer Berlin Heidelberg, 1998.

[2] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd ed., 2026. Online manuscript released January 6, 2026.

[3] scikit-learn Developers, "StandardScaler — scikit-learn documentation." https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.

[4] C. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer series in statistics, Springer, 2009.

[6] scikit-learn Developers, "Classification of text documents using sparse features — scikit-learn documentation." https://scikit-learn.org/stable/auto_examples/text/plot_document_classification_20newsgroups.html.

[7] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proceedings of COMPSTAT'2010* (Y. Lechevallier and G. Saporta, eds.), (Heidelberg), pp. 177–186, Physica-Verlag HD, 2010.

[8] scikit-learn Developers, "compute_class_weight — scikit-learn documentation." https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html.

[9] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing Management*, vol. 45, no. 4, pp. 427–437, 2009.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019.